

FastAPI와 풀스택

템플릿

- FastAPI는 웹 프레임워크로서 웹 서버 구축에 다양한 기능을 제공하며, 이 중 템플릿 엔진 지원이 포함됨
- 이 기능은 HTML 파일 내에서 데이터를 동적으로 처리할 수 있도록 해줌
- FastAPI는 Jinja2라는 강력한 템플릿 엔진을 사용하여 HTML 내에서 파이썬 코드를 사용할 수 있게 해줌
- Jinja2는 FastAPI의 템플릿을 취급할 때 요구되는 중요한 패키지

템플릿

- 템플릿 엔진 설치

- 필수 패키지: FastAPI와 함께 사용하기 위해 jinja2 패키지 필요
- Jinja2 패키지 설치(이미 설치되어 있음)

- `pip show jinja2`
- `pip install jinja2`

- 템플릿 디렉토리 설정

- 프로젝트에 templates 디렉토리를 생성하여 HTML 템플릿 파일들을 관리
 - my_project/
 - main.py # FastAPI 어플리케이션의 메인 파일
 - templates/ # HTML 템플릿 파일 저장소
 - index.html # 웹 페이지 구조를 정의하는 HTML 파일

템플릿

- index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FastAPI Template Example</title>
</head>
<body>

  <h1>Hello, {{ username }}</h1><!--파이썬 코드에서 정의된 변수 값을 HTML에 동적으로 삽입 -->

</body>
</html>
```

FastAPI에서 Jinja2 템플릿 설정하기

- 모듈 импорт

```
from fastapi.templating import Jinja2Templates
```

- FastAPI 어플리케이션 인스턴스 생성 : 이 인스턴스는 라우팅, 미들웨어 등 웹 어플리케이션의 설정을 관리함

```
app = FastAPI()
```

- 템플릿 디렉토리 설정

- Jinja2Templates 클래스의 인스턴스를 생성하고, 템플릿 파일이 위치한 디렉토리를 지정함

```
templates = Jinja2Templates(directory="templates")
```

- directory : 템플릿 파일들이 저장된 디렉토리 경로를 지정. 기본값은 “templates”

템플릿 렌더링

- FastAPI의 템플릿 렌더링 기능
 - FastAPI에서는 HTML 템플릿에 데이터를 삽입하려면 Jinja2Templates 클래스의 TemplateResponse 메소드를 사용함

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/")
def read_root(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"username": "John"}
    )
```

템플릿 렌더링

- FastAPI는 경로(path) 파라미터나, 쿼리(query) 파라미터를 통해 username을 받아 템플릿에 전달할 수 있음
 - 경로 파라미터 사용 예제

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/user/{username}")
def get_user1(request: Request, username: str):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"username": username}
    )
```

- /user/{username} 경로를 통해 username을 받고, 이를 템플릿에 전달함

템플릿 렌더링

- 쿼리 파라미터 사용 예제

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/user")
def get_user2(request: Request, username: str = "John"):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"username": username}
    )
```

- /user 경로와 함께 쿼리 파라미터 username을 받아 템플릿에 전달함

FastAPI와 Jinja2 템플릿 문법

- 변수 출력: `{{ variable_name }}`
 - Jinja2 템플릿에서 변수 값을 HTML 페이지에 출력할 때 사용
 - 파이썬 코드에서 생성한 변수 값을 HTML에 삽입할 수 있음
 - 관련 옵션
 - `/safe` 필터:
 - 이 필터를 사용하면 HTML 태그가 포함된 문자열을 웹 페이지에서 안전하게 렌더링할 수 있음
 - HTML 태그가 포함된 문자열을 그대로 렌더링하려면 이 필터를 사용함
 - 단, 사용자 입력을 그대로 렌더링할 때는 XSS(Cross-Site Scripting)등의 보안 문제를 유발할 수 있으므로 신중하게 사용해야 됨
- 주석: `{# comment #}`
 - 코드 내에서 설명이나 메모를 남기기 위해 사용
 - 주석은 렌더링되지 않아 사용자에게는 보이지 않음
 - 옵션
 - 다중 라인 주석:
 - 주석은 시작 태그 `{#` 와 끝 태그 `#}` 사이에 위치함. 이 안에 한 라인 또는 여러 라인을 넣어 다중 라인 주석을 만들 수 있음

제어문: {%if ... %} ... {%endif%}

- 제어문을 사용하면 렌더링되는 HTML 콘텐츠를 동적으로 변경할 수 있음
- Jinja2의 제어문은 FastAPI 코드에서 받은 변수의 값을 기반으로 HTML을 조건적으로 렌더링하는 데 사용됨
- 다양한 제어문 문법
 - IF Statement: {%if...%}...{%endif%}
 - Else Statement: {%else%}...
 - Elseif Statement: {%elif... %}...

반복문: {%for item in items%}...{%endfor%}

- 반복문은 Jinja2에서 동적으로 여러 요소를 렌더링할 때 자주 사용
- FastAPI에서 Python 리스트나 다른 반복 가능한 객체를 HTML 템플릿에 전달할 수 있고, Jinja2의 {% for %} 구문을 사용해서 이러한 요소를 렌더링할 수 있음

Jinja2 템플릿의 상속

- FastAPI와 Jinja의 상속 기능

- 상속 사용 방법: {% extends 'base.html'%}
- Jinja2 템플릿에서 상속을 사용하면, 공통적인 HTML 구조를 base.html에 정의하고 다른 HTML 템플릿에서 이를 상속받을 수 있음
- 이 방법은 웹사이트의 헤더, 푸터 등 반복적인 부분을 중복 작성하지 않고 효율적으로 관리할 수 있게 해 줌

- 상속의 장점

- 코드 재사용 향상: 공통 요소를 한 곳에 모아 관리함으로써 코드의 중복을 줄이고 재사용성을 높일 수 있음
- 유지 보수 용이: 공통 요소의 변경이 필요한 경우, 한 곳에서 수정하면 모든 상속받은 템플릿에 자동으로 반영됨으로 유지 보수가 용이
- 일관된 레이아웃 유지: 사이트 전반에 걸쳐 일관된 레이아웃과 스타일을 유지할 수 있음

고급 템플릿 기능

- Jinja2 템플릿의 include 태그

- include 태그의 주요 기능

- {% include '파일명.html'%} : 지정된 HTML 파일을 현재 템플릿 파일에 포함시키는 기능 제공
 - 이를 통해 템플릿의 특정 부분을 다른 파일로 분리하고, 필요한 곳에서 재사용할 수 있음

- include 태그의 장점

- 코드 재사용성 향상 : 헤더, 푸터, 네비게이션 바 등과 같은 반복되는 요소를 별도의 파일로 분리하여 여러 페이지에서 재사용할 수 있음
 - 유지 보수 용이 : 공통 요소의 변경이 필요한 경우, 한 곳에서 수정하면 모든 페이지에 자동으로 반영되어 유지 보수가 용이
 - 가독성 및 구조화 향상 : 템플릿을 작은 단위로 분리함으로써 가독성이 높아지고, 코드 구조가 명확해짐

- Jinja2의 include 태그를 활용하면 FastAPI를 사용한 웹 개발에서 보다 깔끔하고 효율적인 코드 작성이 가능. 이를 통해 개발자는 코드 중복을 줄이고, 유지 보수를 쉽게 할 수 있는 강력한 웹 어플리케이션을 구축할 수 있음

정적 파일(Static Files) 관리

- FastAPI의 정적 파일 관리 기능

- 정적 파일이란?

- 웹 개발에서 HTML, CSS, JavaScript, 이미지 파일과 같은 변하지 않는 파일들을 의미
 - 서버에서 변경없이 그대로 브라우저에 전달되는 파일들로 구성
 - 마운트(Mount) : FastAPI에서는 특정 경로(URL)에 폴더나 다른 어플리케이션을 연결하는 작업

- 사용 시 주의 사항

- 보안 문제 : 정적 파일을 제공할 때는 보안에 주의해야 함. 잘못 설정하면 민감한 데이터가 노출될 수 있음
 - 최적화 : 대량의 정적 파일을 제공할 때는 웹 서버(Nginx, Apache 등)를 이용하는 것이 성능상 더 효율적일 수 있음

- FastAPI에서 정적 파일 관리 기능은 웹 어플리케이션 개발에서 필수적인 요소로, 이를 통해 웹 페이지에 필요한 다양한 리소스를 효과적으로 관리하고 제공할 수 있음